System Analysis & Design

1. Problem Statement & Objectives – Define the problem being solved and project goals:

- **Problem Statement**

Users often struggle to manage their daily tasks and maintain consistent habits due to the lack of a clear structure and limited visibility into their daily and weekly progress. Most existing solutions are either too complex or focus only on tasks without supporting long-term routine building.

LifeMap aims to solve this problem by providing a simple, structured, and user-friendly system that combines daily task management, habit tracking, and progress monitoring in one place.

- Project Objectives

- Provide a clear and simple way to manage daily tasks
- Enable users to build and maintain daily routines and habits
- Offer visual insights into user progress and consistency
- Reduce friction in daily planning and tracking
- Encourage continuous self-improvement through consistent usage

---

- Use Case Diagram (Textual Representation)

Use Cases

- Create Task
- Update Task Status (To Do / In Progress / Done)
- View Daily Tasks
- Create Habit
- Track Habit Completion
- Set Reminders
- View Statistics
- Manage Profile and Settings

Use Case Descriptions

- **Create Task:** The user enters task details and adds it to the daily task list.
- **Update Task Status:** The user changes the task state between To Do, In Progress, and Done.
- **View Daily Tasks:** The user views a list of tasks assigned to the current day.
- **Create Habit:** The user defines a new habit with repetition days and optional reminders.
- **Track Habit Completion:** The user marks a habit as completed for the current day.
- **Set Reminders:** The user enables reminders for habits to maintain consistency.
- **View Statistics:** The user views progress data such as completed tasks and habit consistency.
- **Manage Profile and Settings:** The user updates personal information and application preferences

Functional Requirements

- The system allows users to create, edit, and delete tasks
- The system allows users to update task status
- The system allows users to create and manage habits
- The system allows users to track daily habit completion
- The system provides reminders for habits
- The system displays statistics and progress insights
- The system stores data locally using a database

Non-Functional Requirements

- The application should load each screen in less than 2 seconds
- The application should work offline using local storage
- The user interface should be simple and intuitive
- The application should be stable with minimal crashes
- The application should require minimal steps to complete daily actions

---

Software Architecture

The application follows the MVVM (Model-View-ViewModel) architecture pattern.

Presentation Layer

- Built using Jetpack Compose
- Handles UI rendering and user interactions
- Uses ViewModels for state management

Domain Layer

- Contains business logic and use cases
- Acts as a bridge between UI and data layers

Data Layer

- Includes Repository pattern
- Uses Room database for local data storage
- Handles data retrieval and persistence

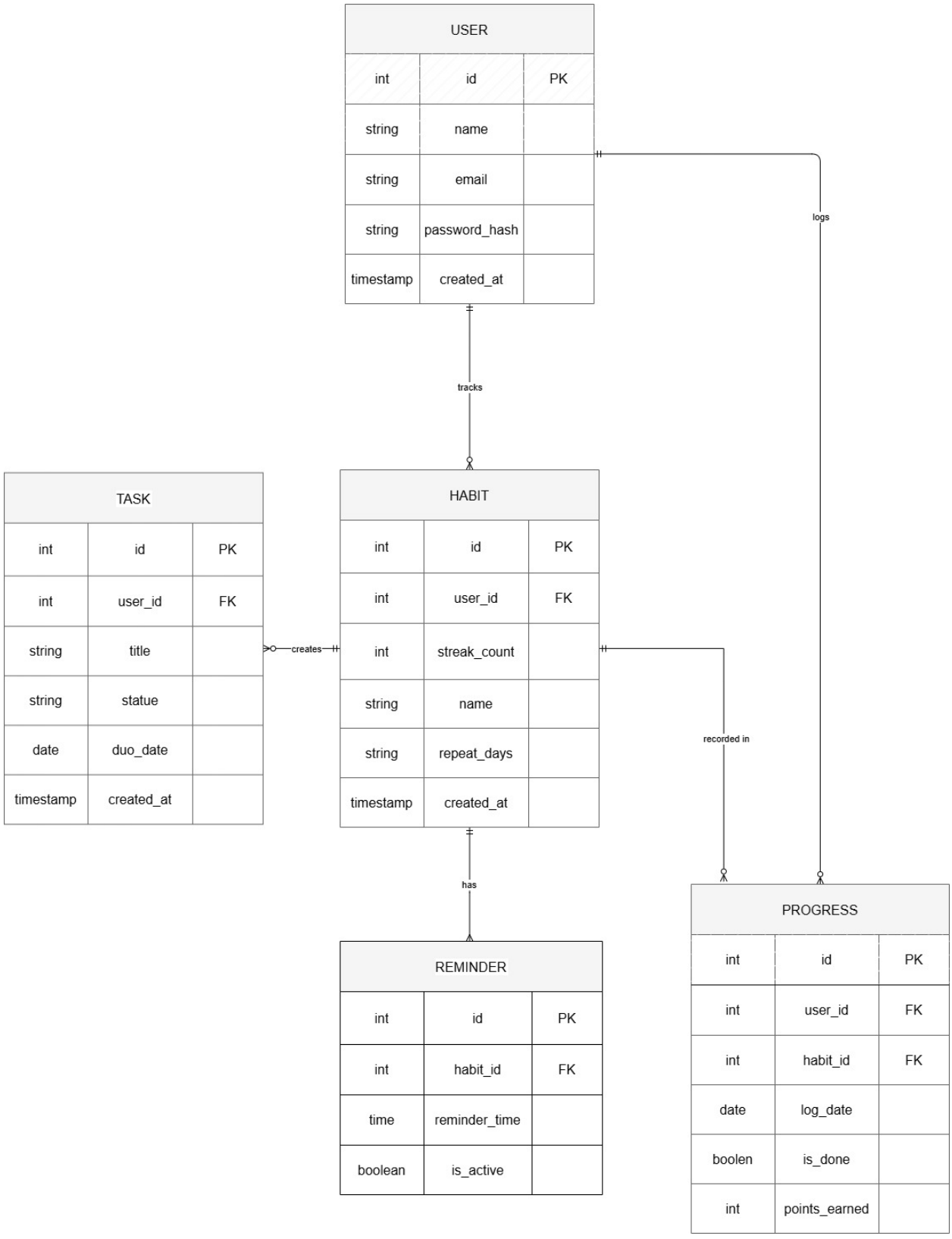
Data Flow

UI → ViewModel → Repository → Local Database

This structure ensures separation of concerns, maintainability, and scalability of the application.

2. Database design and modeling:

- ERD:


USER
int id PK
string name
string email
string password_hash
timestamp created_at
logs
tracks
TASK
int id PK
int user_id FK
string title
string statue
date duo_date
timestamp created_at
creates
HABIT
int id PK
int user_id FK
int streak_count
string name
string repeat_days
timestamp created_at
recorded in
has
REMINDER
int id PK
int habit_id FK
time reminder_time
boolean is_active
PROGRESS
int id PK
int user_id FK
int habit_id FK
date log_date
boolen is_done
int points_earned

- **Physical Schema:**

USER

```
CREATE TABLE USER (
  id INT PRIMARY KEY AUTO_INCREMENT,
  name VARCHAR(100) NOT NULL,
  email VARCHAR(150) NOT NULL UNIQUE,
  password_hash TEXT NOT NULL,
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```

TASK

```
CREATE TABLE TASK (
  id INT PRIMARY KEY AUTO_INCREMENT,
  user_id INT NOT NULL,
  title VARCHAR(200) NOT NULL,
  status VARCHAR(50) NOT NULL,
  due_date DATE,
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  FOREIGN KEY (user_id) REFERENCES USER(id)
);
```

HABIT

```
CREATE TABLE HABIT (
  id INT PRIMARY KEY AUTO_INCREMENT,
  user_id INT NOT NULL,
  name VARCHAR(100) NOT NULL,
  repeat_days VARCHAR(50) NOT NULL,
  streak_count INT DEFAULT 0,
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
```

```
  FOREIGN KEY (user_id) REFERENCES USER(id)
);
```

REMINDER

```
CREATE TABLE REMINDER (
  id INT PRIMARY KEY AUTO_INCREMENT,
  habit_id INT NOT NULL,
  reminder_time TIME NOT NULL,
  is_active BOOLEAN DEFAULT TRUE,
  FOREIGN KEY (habit_id) REFERENCES HABIT(id)
);
```

PROGRESS

```
CREATE TABLE PROGRESS (
  id INT PRIMARY KEY AUTO_INCREMENT,
  user_id INT NOT NULL,
  habit_id INT NOT NULL,
  log_date DATE NOT NULL,
  is_done BOOLEAN DEFAULT FALSE,
  points_earned INT DEFAULT 0,
  FOREIGN KEY (user_id) REFERENCES USER(id),
  FOREIGN KEY (habit_id) REFERENCES HABIT(id)
);
```

- ## Normalization:

  The database design follows 3NF. Each table has a primary key, no repeating groups exist (1NF), all attributes depend on the full primary key (2NF), and no transitive dependencies exist between non-key attributes (3NF).

- Logical Shema:

USER

Column Name	Data Type	Constraints
id	INT	Primary Key
name	VARCHAR	Not Null
email	VARCHAR	Not Null, Unique
password_hash	TEXT	Not Null
created_at	TIMESTAMP	Default Now

TASK

Column Name	Data Type	Constraints
id	INT	Primary Key
user_id	INT	Foreign Key USER(id)
title	VARCHAR	Not Null
status	VARCHAR	Not Null
due_date	DATE	Nullable
created_at	TIMESTAMP	Default Now

HABIT

Column Name	Data Type	Constraints
id	INT	Primary Key
user_id	INT	Foreign Key USER(id)
name	VARCHAR	Not Null
repeat_days	VARCHAR	Not Null
streak_count	INT	Default 0
created_at	TIMESTAMP	Default Now

REMINDER

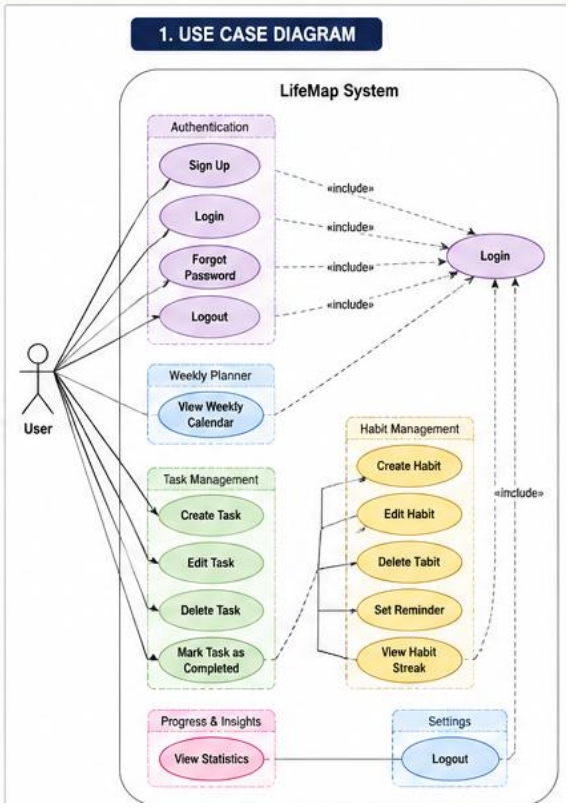
Column Name	Data Type	Constraints
id	INT	Primary Key
habit_id	INT	Foreign Key HABIT(id)
reminder_time	TIME	Not Null
is_active	BOOLEAN	Default True

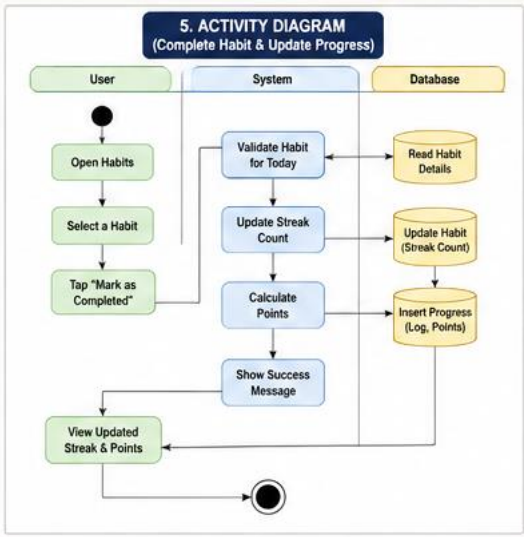
PROGRESS

Column Name	Data Type	Constraints
id	INT	Primary Key
user_id	INT	Foreign Key USER(id)

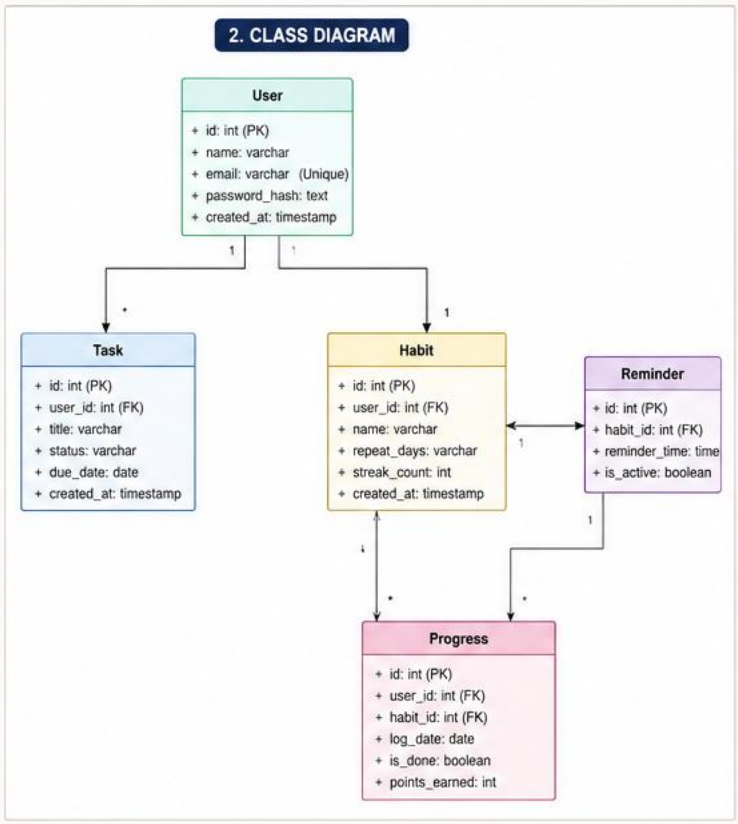
habit_id	INT	Foreign Key HABIT(id)
log_date	DATE	Not Null
is_done	BOOLEAN	Default False
points_earned	INT	Default 0

3. DataFlow & System Behvior:

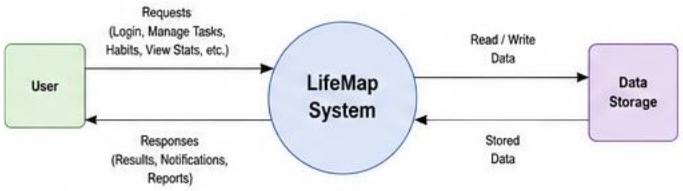




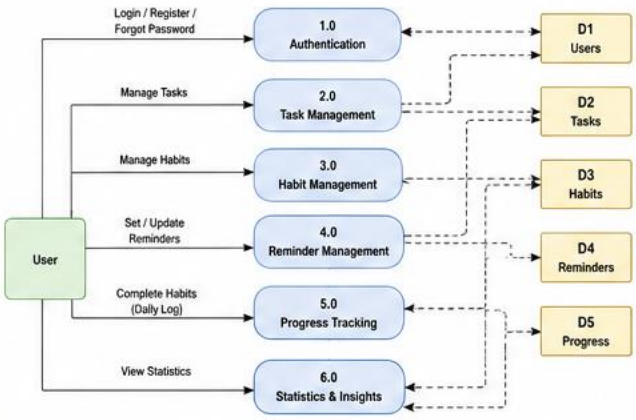
2. CLASS DIAGRAM

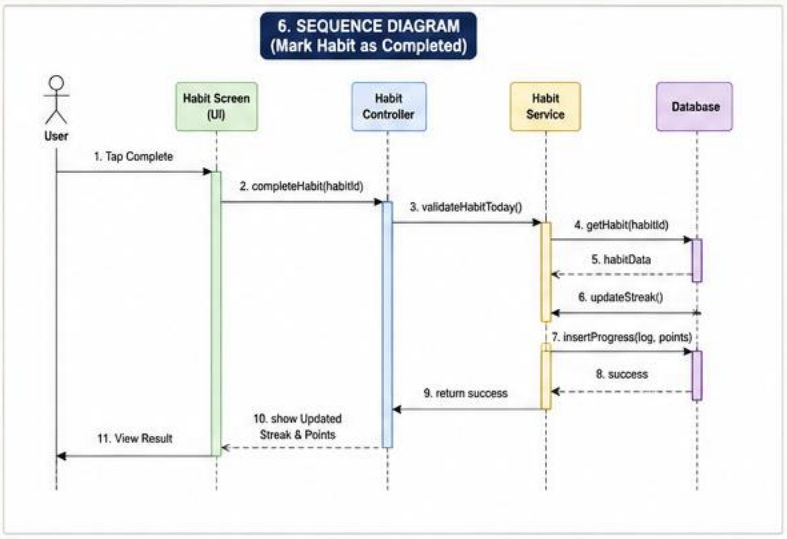


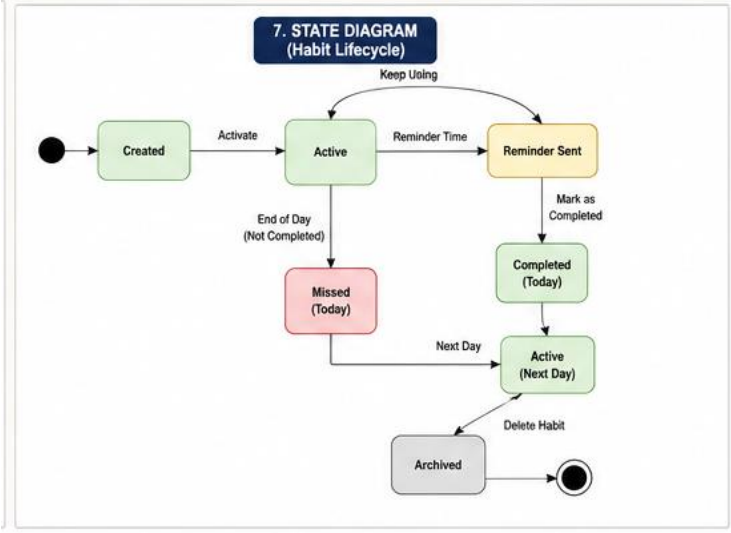
3. DFD - CONTEXT DIAGRAM



4. DFD - LEVEL 1







4. UI& UX Modeling :

https://www.figma.com/make/t0hNCoPXW720dW3RZwl84f/LifeMap-Productivity-App-Design?p=f&t=Id9XnIVemtFuxGVY-0

5. System Deployment & Integration:

5.1 Technology Stack

LifeMap is built as a native Android application using a serverless cloud backend, which keeps the architecture lightweight while providing real-time synchronization, authentication, and push notifications out of the box. The table below summarizes the technologies used across each layer of the system.

Layer	Technology	Purpose
Programming Language	Kotlin	Primary language for native Android development
IDE	Android Studio	Development, debugging, and UI design environment
Architecture Pattern	MVVM (Model-View-ViewModel)	Separates UI, business logic, and data for maintainability
UI Framework	Android Jetpack (XML / Jetpack Compose)	Building responsive, modern UI screens
Local Database	Room (SQLite abstraction)	Offline caching of goals, habits, and tasks
Backend / Cloud	Firebase Platform (Google Cloud)	Serverless backend for auth, data, and messaging
Remote Database	Cloud Firestore	Real-time NoSQL storage synced across devices
Authentication	Firebase Authentication	Secure user sign-up, login, and session handling
Notifications	Firebase Cloud Messaging (FCM)	Push reminders for habits, goals, and tasks

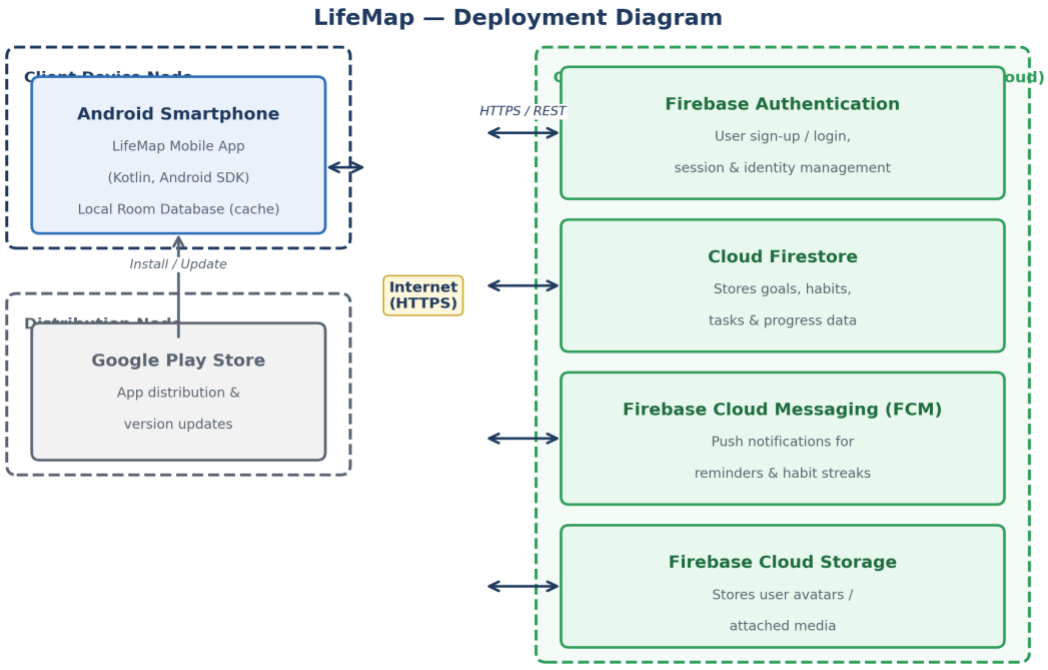
Layer	Technology	Purpose
File Storage	Firebase Cloud Storage	Stores user avatars and attached media
Version Control	Git & GitHub	Source code management and team collaboration
UI/UX Design Tool	Figma	Wireframing and prototyping screens before development

Justification of Key Choices

- **Kotlin** was chosen over Java for its concise, null-safe syntax and full official support by Google for Android development.
- **Firebase** (Firestore, Authentication, FCM, Cloud Storage) removes the need to build and host a custom backend server, which suits the scope and timeline of a graduation project while still demonstrating real-world cloud integration.
- **Room (local database)** provides offline-first behavior, so users can view and update their goals, habits, and tasks even without an internet connection; changes sync to Firestore once connectivity is restored.
- **MVVM architecture** (MVVM) cleanly separates UI code from business logic, making the codebase easier to test, maintain, and extend by team members working in parallel.

5.2 Deployment Diagram

The deployment diagram below shows how LifeMap's software components are physically distributed across hardware nodes: the user's Android device, the Google Play Store distribution channel, and the Firebase cloud platform that hosts all backend services.

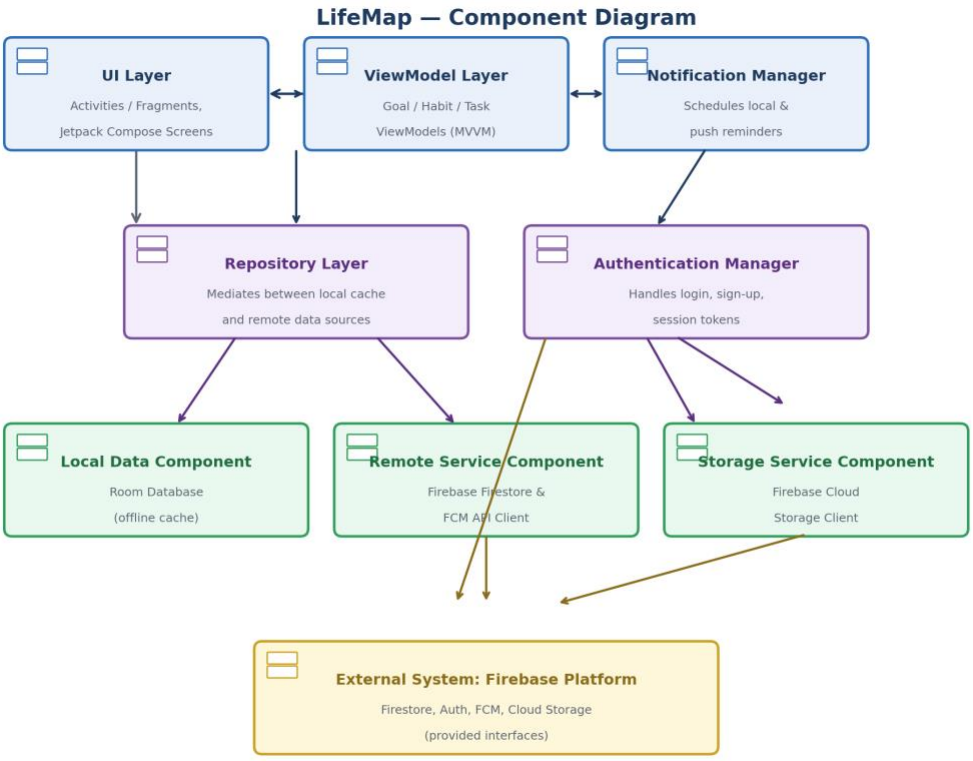


Description

- **Android Smartphone** (Client Node): Runs the LifeMap Android app together with a local Room database that caches data for offline use.
- **Google Play Store** (Distribution Node): Hosts the published APK/AAB and delivers installations and updates to user devices.
- **Firebase Platform** (Cloud Server Node): A managed Google Cloud environment hosting four services the app depends on:
  - Firebase Authentication — manages user accounts and secure sessions.
  - Cloud Firestore — stores and synchronizes goals, habits, tasks, and progress data in real time.
  - Firebase Cloud Messaging (FCM) — delivers push notifications for reminders and streaks.
  - Firebase Cloud Storage — stores user profile pictures and other media attachments.
- **Communication:** All communication between the Android client and Firebase services occurs over encrypted HTTPS connections, ensuring data security in transit.

5.3 Component Diagram

The component diagram below illustrates the high-level software components inside the LifeMap application and the dependencies between them, grouped into presentation, logic/mediation, and data layers, plus the external Firebase system they connect to.



Description

- **UI Layer:** Screens built with Activities/Fragments (or Jetpack Compose) that display goals, habits, and tasks to the user, and forward user actions upward.
- **ViewModel Layer:** Holds UI-related state and exposes it to the UI Layer via observable data holders, following the MVVM pattern.
- **Notification Manager:** Schedules and displays local and push reminders for habits and deadlines.
- **Repository Layer:** Acts as the single source of truth for data, deciding whether to serve data from the Local Data Component (Room) or fetch/sync it from the Remote Service Component (Firestore).
- **Authentication Manager:** Handles all sign-up, login, and session logic by communicating with Firebase Authentication.
- **Local Data Component:** Wraps the Room database for fast, offline-first access to cached data.
- **Remote Service Component:** Wraps the Firebase Firestore SDK and FCM client for remote data sync and notifications.
- **Storage Service Component:** Wraps the Firebase Cloud Storage SDK for uploading and retrieving media files.

- **External System:** The Firebase Platform is an external system providing well-defined interfaces (APIs/SDKs) that the app's data-layer components depend on, rather than a component the team builds itself.